



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

E-Vehicle Charging Management System for a Smart University Campus

Bagya Venkatesh(192324263) :

<https://github.com/bagya1275/Java-programming/tree/main/assignment>

Subiksha U (192424135):

<https://github.com/subixu/JAVA---CSA0905-/tree/main/ASSIGNMENT%20DOC>

Hema Priya T(192524467):

[192524467-JAVA/ASSIGNMENT at main · 192524467simats-wq/192524467-JAVA](#)

PROBLEM STATEMENT

Develop a GUI-based Java application using AWT/Swing and JDBC to manage an E-Vehicle Charging Facility on a university campus. The system should manage users, electric vehicles, charging stations, charging points, reservations, charging sessions, energy consumption, and payments.

It should provide charging-point availability, slot reservation, check-in/check-out, charging cost calculation, payment management, and reports. The application should use MySQL with JDBC for database operations and include input validation, conflict checking, exception handling, and proper database relationships to ensure efficient and reliable charging management.

OBJECTIVE

To develop a Java Swing-based system that efficiently manages EV charging, reservations, billing, payments, and reports on a university campus, while presenting the operational information through a clear dashboard and visualization interface.

PSEUDO CODE:

```
BEGIN
    DISPLAY "EV VEHICLE CHARGING MANAGEMENT SYSTEM"

1. APPLICATION STARTUP
    LOAD / INITIALIZE application data
    CREATE the Java Swing user interface
    CONFIGURE window, sidebar, top bar and main body
    DISPLAY Dashboard

2. DASHBOARD
    CALCULATE total charging points
    CALCULATE available points
    CALCULATE occupied points
    CALCULATE total energy delivered
    DISPLAY KPI cards
    DISPLAY live point-status cards
    DISPLAY quick actions

3. USER / VEHICLE MANAGEMENT
    IF user selects Users or Vehicles THEN
        DISPLAY existing records
        ALLOW registration of a new user or vehicle
        VALIDATE required input
        IF input is valid THEN
            CREATE the corresponding record
            REFRESH the page
        ELSE
            DISPLAY validation warning
        END IF
    END IF
```

```

4. CHARGING STATION MANAGEMENT
  IF user selects Charging Stations THEN
    DISPLAY station name
    DISPLAY campus location
    DISPLAY charging point
    DISPLAY connector type
    DISPLAY current status
    ALLOW status change
    IF user adds a station THEN
      INPUT station name and campus location
      INPUT charging point type
      VALIDATE required fields
      CREATE station and point record
      REFRESH Stations page
    END IF
  END IF

5. CHARGING POINT MANAGEMENT
  FOR each charging station
    FOR each charging point
      DISPLAY point ID
      DISPLAY connector type
      DISPLAY status
      IF point is available THEN
        DISPLAY "AVAILABLE"
      ELSE IF point is occupied THEN
        DISPLAY "OCCUPIED"
      ELSE IF point is reserved THEN
        DISPLAY "RESERVED"
      ELSE
        DISPLAY "MAINTENANCE"
      END IF
    END FOR
  END FOR

6. RESERVATION
  IF no vehicles exist THEN
    DISPLAY "Register a vehicle first"
    STOP
  END IF

  FIND usable charging points
  IF no usable point exists THEN
    DISPLAY "No usable charging points available"
    STOP
  END IF

  SELECT vehicle
  SELECT charging point
  INPUT reservation slot
  IF user confirms THEN
    CREATE Reservation record
    ADD reservation to reservation list
    CHANGE selected point status to "RESERVED"
    SAVE application state
    REFRESH Reservations page
  END IF

```

```

7. START / RECORD CHARGING SESSION
  IF no vehicles exist THEN
    DISPLAY "Register a vehicle first"
    STOP
  END IF

  BUILD list of usable charging points
  SELECT vehicle and charging point
  INPUT energyConsumed
  INPUT charging duration

  IF energyConsumed <= 0 OR duration <= 0 THEN
    DISPLAY validation warning
    STOP
  END IF

  selectedPoint.status ← "AVAILABLE"
  cost ← energyConsumed × 8.50

  CREATE Session(
    vehicle,
    point,
    energyConsumed,
    duration,
    cost,
    "Completed"
  )

  ADD Session to session list

  CREATE Payment(
    session,
    cost,
    "Paid"
  )

  ADD Payment to payment list
  SAVE application state
  REFRESH Charging Sessions page

8. COST CALCULATION
  INPUT energyConsumed
  SET chargingRate ← 8.50
  totalCost ← energyConsumed × chargingRate
  DISPLAY totalCost

9. PAYMENT
  IF a charging session is completed THEN
    RETRIEVE session amount
    CREATE payment record
    SET payment status ← "Paid"
    DISPLAY payment information
  END IF

```

```

10. ANALYTICS AND REPORTS
    sessions ← number of recorded sessions
    energy ← SUM(session.kWh)
    revenue ← SUM(session.cost)

    COUNT charging points by status:
        Available
        Occupied
        Reserved
        Maintenance

    CALCULATE utilization
    DISPLAY session count
    DISPLAY total energy
    DISPLAY total revenue
    DISPLAY utilization
    DRAW point-distribution chart
    DISPLAY management insights

11. VALIDATION
    FOR every user input
        CHECK required fields
        CHECK numeric values
        CHECK vehicle / registration values
        CHECK reservation selection
        IF invalid THEN
            DISPLAY appropriate warning
            ASK user to enter again
        END IF
    END FOR

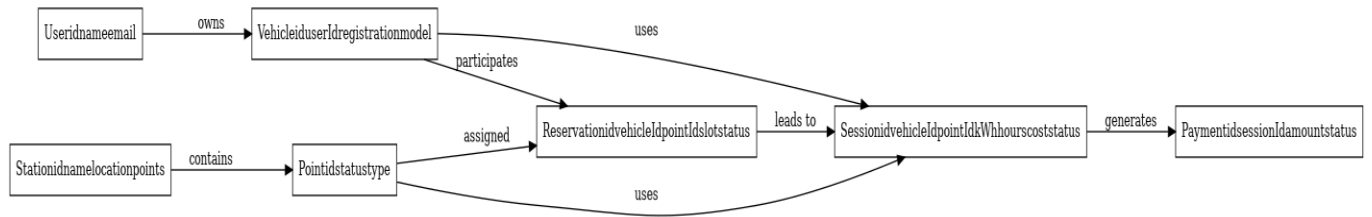
12. EXCEPTION / ERROR HANDLING
    TRY application operations
        PERFORM requested operation
    CATCH invalid input
        DISPLAY validation warning
    CATCH unavailable charging point
        DISPLAY availability message
    CATCH unexpected exception
        DISPLAY error message
    END TRY

13. NAVIGATION
    IF user selects another page THEN
        CLEAR current body
        LOAD requested page
        REFRESH interface
    END IF

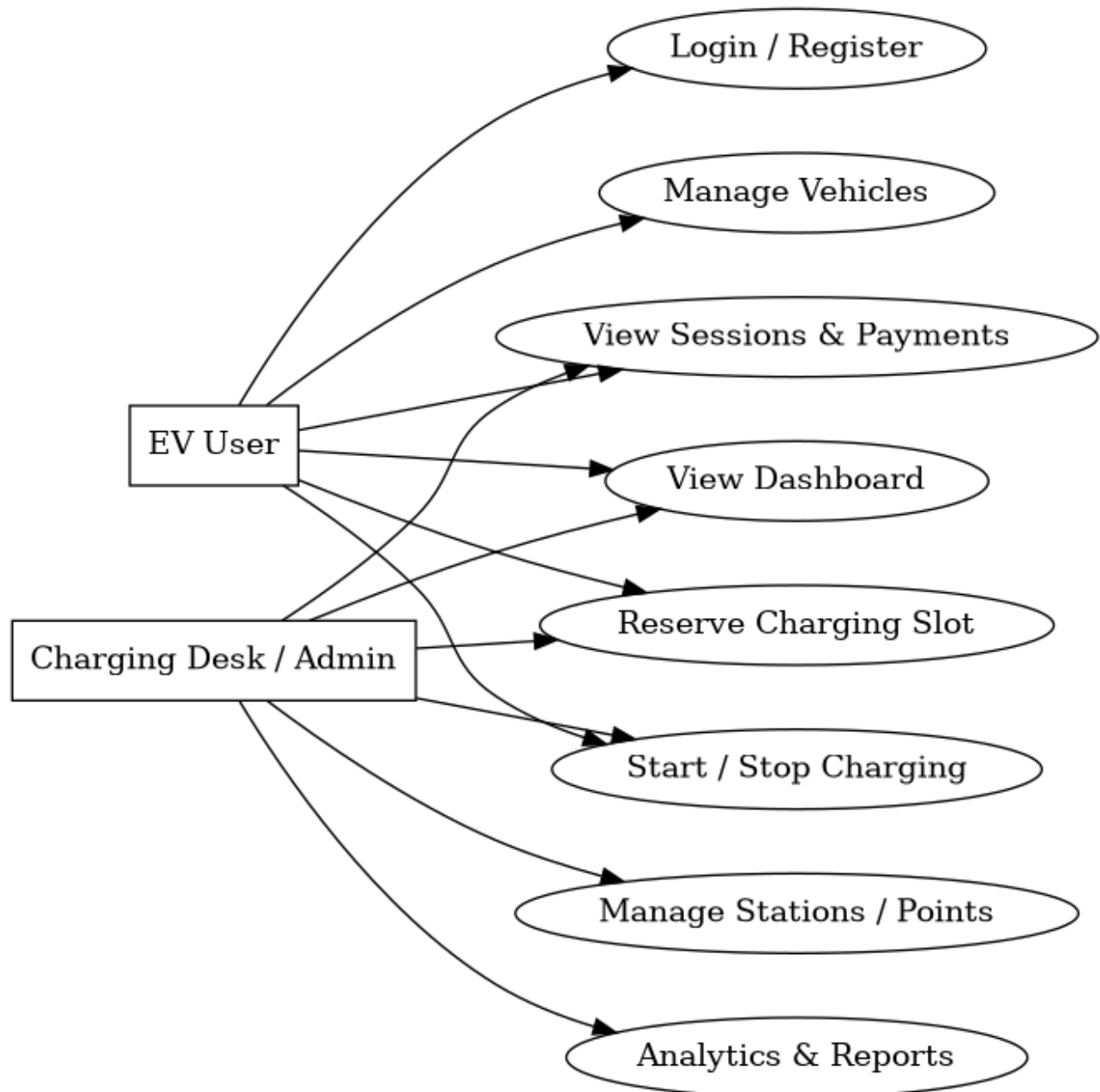
14. EXIT
    IF user closes application THEN
        SAVE supported local state
        CLOSE application
    END IF
END

```

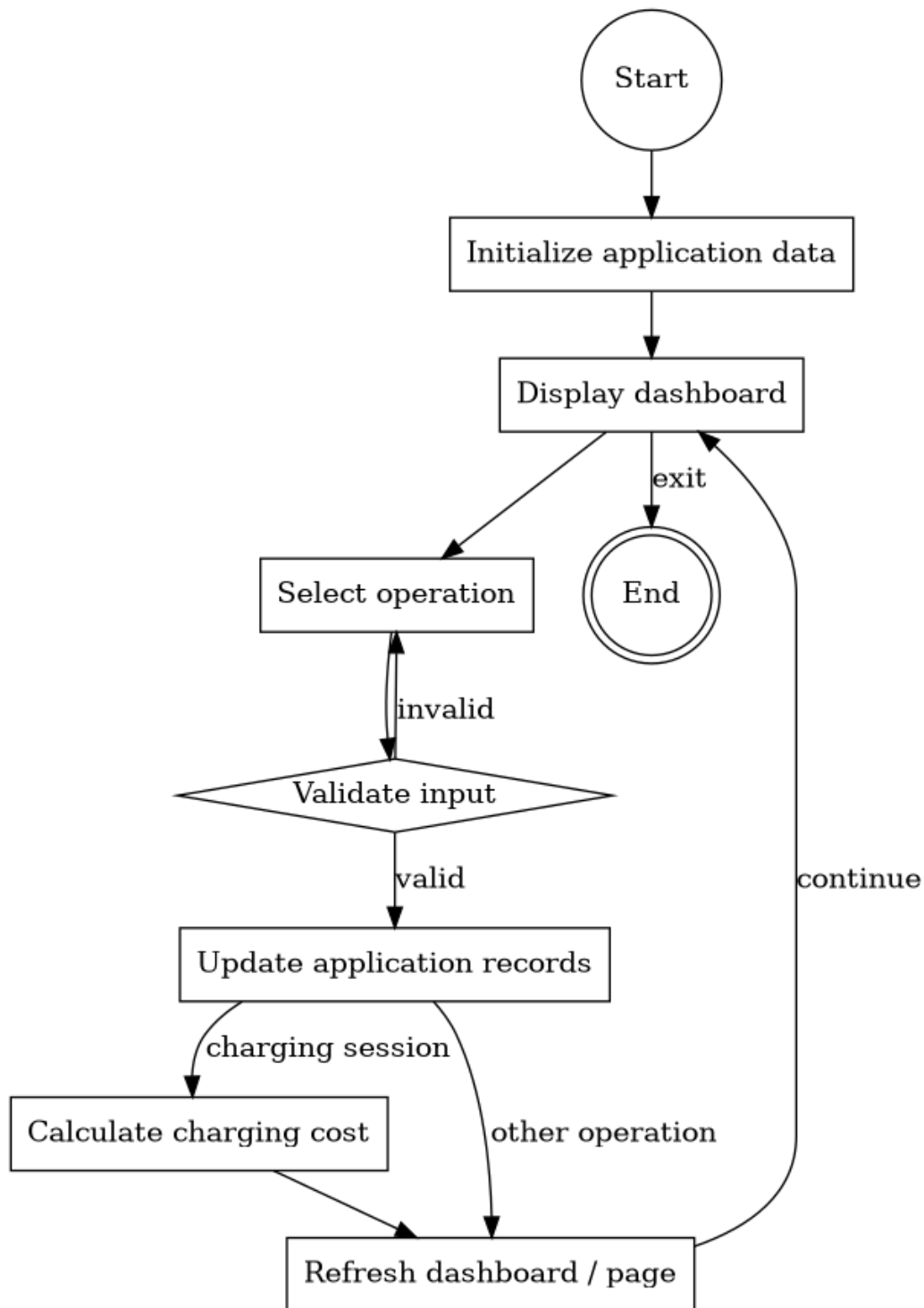
CLASS DIAGRAM



USECASE DIAGRAM



ACTIVITY DIAGRAM



DATA MODEL

Entity	Important Attributes	Relationship / Use
User	id, name, email	Owns one or more registered vehicles
Vehicle	id, userId, registration, model	Belongs to a user; participates in reservations and sessions
Station	id, name, location, points	Contains charging points
Point	id, status, type	Represents an individual charging point
Reservation	id, vehicleId, pointId, slot, status	Connects a vehicle to a charging point and slot
Session	id, vehicleId, pointId, kWh, hours, cost, status	Records charging activity and calculated cost
Payment	id, sessionId, amount, status	Represents payment generated from a completed session

IMPLEMENTATION CODE:

MAIN.JAVA

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.io.*;
import java.util.*;

public class Main {

    static final double TARIFF = 8.50;

    static class User {
        int id;
        String name;
        String email;

        User(int id, String name, String email) {
            this.id = id;
            this.name = name;
            this.email = email;
        }
    }

    static class Vehicle {
        int id;
        int userId;
        String reg;
        String model;

        Vehicle(int id, int userId, String reg, String model) {
            this.id = id;
            this.userId = userId;
            this.reg = reg;
            this.model = model;
        }
    }

    static class Point {
        int id;
        String status;
        String type;

        Point(int id, String status, String type) {
            this.id = id;
            this.status = status;
            this.type = type;
        }
    }

    static class Station {
        int id;
        String name;
        String location;
        List<Point> points = new ArrayList<>();

        Station(int id, String name, String location) {
            this.id = id;
            this.name = name;
            this.location = location;
        }
    }
}
```

```

static class Reservation {
    int id;
    int vehicleId;
    int pointId;
    String slot;
    String status;

    Reservation(int id, int vehicleId, int pointId,
                String slot, String status) {
        this.id = id;
        this.vehicleId = vehicleId;
        this.pointId = pointId;
        this.slot = slot;
        this.status = status;
    }
}

static class Session {
    int id;
    int vehicleId;
    int pointId;
    double kwh;
    double hours;
    double cost;
    String status;

    Session(int id, int vehicleId, int pointId,
            double kwh, double hours, String status) {
        this.id = id;
        this.vehicleId = vehicleId;
        this.pointId = pointId;
        this.kwh = kwh;
        this.hours = hours;
        this.cost = kwh * TARIFF;
        this.status = status;
    }
}

static class Payment {
    int id;
    int sessionId;
    double amount;
    String status;

    Payment(int id, int sessionId, double amount,
            String status) {
        this.id = id;
        this.sessionId = sessionId;
        this.amount = amount;
        this.status = status;
    }
}

public static void main(String[] args) {
    load();
    SwingUtilities.invokeLater(Main::build);
}

static void build() {
    try {
        UIManager.setLookAndFeel(
            UIManager.getCrossPlatformLookAndFeelClassName()
        );
    } catch (Exception ignored) {
    }

    frame = new JFrame(
        "EV ChargeHub | Smart Campus Charging"
    );

    frame.setDefaultCloseOperation(

```

```
        JFrame.EXIT_ON_CLOSE
    );
    frame.setSize(1250, 790);
    frame.setMinimumSize(
        new Dimension(1050, 680)
    );
    frame.setLocationRelativeTo(null);
```

```

        JPanel root = new JPanel(
            new BorderLayout()
        );
        root.setBackground(BG);

        root.add(
            sidebar(),
            BorderLayout.WEST
        );

        JPanel main = new JPanel(
            new BorderLayout()
        );
        main.setBackground(BG);

        main.add(
            topbar(),
            BorderLayout.NORTH
        );

        body = new JPanel(
            new BorderLayout()
        );
        body.setBackground(BG);
        body.setBorder(
            new EmptyBorder(
                20, 26, 26, 26
            )
        );

        main.add(
            body,
            BorderLayout.CENTER
        );

        root.add(
            main,
            BorderLayout.CENTER
        );

        frame.setContentPane(root);
        frame.setVisible(true);

        show("Dashboard");
    }

    static void dashboard() {
        int totalPoints = 0;
        int available = 0;
        int occupied = 0;
        double energy = getEnergy();

        for (Station s : stations) {
            for (Point p : s.points) {
                totalPoints++;

                if (p.status.equals("AVAILABLE"))
                    available++;

                if (p.status.equals("OCCUPIED"))
                    occupied++;
            }
        }

        cards.add(
            card(
                "Available",
                String.valueOf(available),
                available + " ready to charge",
                GREEN
            )
        );
    }

```

```
);

cards.add(
    card(
        "Active / Occupied",
        String.valueOf(occupied),
        "Currently in use",
        ORANGE
    )
);

cards.add(
    card(
        "Energy Delivered",
        String.format("%.1f kWh", energy),
        "Recorded sessions",
        PURPLE
    )
);
}
```

```

static void changeStatus(int pointId) {
    Point p = findPoint(pointId);

    if (p == null)
        return;

    String[] statuses = {
        "AVAILABLE",
        "OCCUPIED",
        "RESERVED",
        "MAINTENANCE"
    };

    String selected =
        (String) JOptionPane.showInputDialog(
            frame,
            "Select new status:",
            "Change Point Status",
            JOptionPane.PLAIN_MESSAGE,
            null,
            statuses,
            p.status
        );

    if (selected == null)
        return;

    p.status = selected;
    save();
}

static void addStation() {
    JTextField name = new JTextField();
    JTextField location = new JTextField();

    JComboBox<String> type =
        new JComboBox<>() {
            new String[] {
                "AC Type-2",
                "DC Fast",
                "CCS2"
            }
        };

    JPanel form = new JPanel(
        new GridLayout(0, 2, 8, 8)
    );

    form.add(new JLabel("Station Name"));
    form.add(name);
    form.add(new JLabel("Campus Location"));
    form.add(location);
    form.add(new JLabel("Connector Type"));
    form.add(type);

    int result =
        JOptionPane.showConfirmDialog(
            frame,
            form,
            "Add Charging Station",
            JOptionPane.OK_CANCEL_OPTION
        );

    if (result != JOptionPane.OK_OPTION)
        return;

    if (name.getText().trim().isEmpty()
        || location.getText().trim().isEmpty()) {

        JOptionPane.showMessageDialog(
            frame,

```

```
        "Please enter all required details."
    );
    return;
}

Station station = new Station(
    nextStationId(),
    name.getText().trim(),
    location.getText().trim()
);

station.points.add(
    new Point(
        nextPointId(),
        "AVAILABLE",
        type.getSelectedItem().toString()
    )
);

stations.add(station);
save();
show("Stations");
}
```

OUTPUT / RESULTS

The following screenshots document the working Java Swing interface. The pages use the same dashboard and navigation design supplied in the student's project and show the principal operational screens used during demonstration.

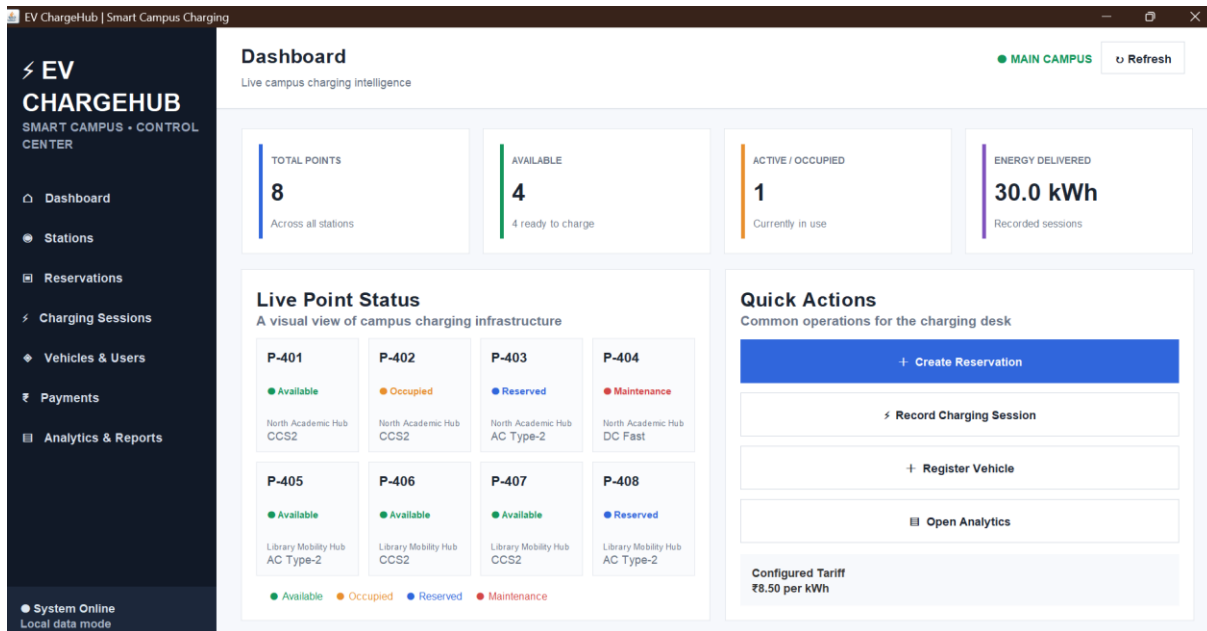


Figure 1. Dashboard – KPI cards, live point status and quick actions.

EV ChargeHub | Smart Campus Charging

EV CHARGEHUB

SMART CAMPUS • CONTROL CENTER

Dashboard

Stations

Reservations

Charging Sessions

Vehicles & Users

Payments

Analytics & Reports

System Online

Local data mode

Charging Stations

Infrastructure, point status and availability

MAIN CAMPUS

Refresh

Station Directory

Select a point to update its operational state

Add Station

Station	Location	Point	Connector	Status	Action
North Academic Hub	Academic Block	P-401	CCS2	Available	Change
North Academic Hub	Academic Block	P-402	CCS2	Occupied	Change
North Academic Hub	Academic Block	P-403	AC Type-2	Reserved	Change
North Academic Hub	Academic Block	P-404	DC Fast	Maintenance	Change
Library Mobility Hub	Central Campus	P-405	AC Type-2	Available	Change
Library Mobility Hub	Central Campus	P-406	CCS2	Available	Change
Library Mobility Hub	Central Campus	P-407	CCS2	Available	Change
Library Mobility Hub	Central Campus	P-408	AC Type-2	Reserved	Change

Figure 2. Charging Stations – station directory with point and status information.

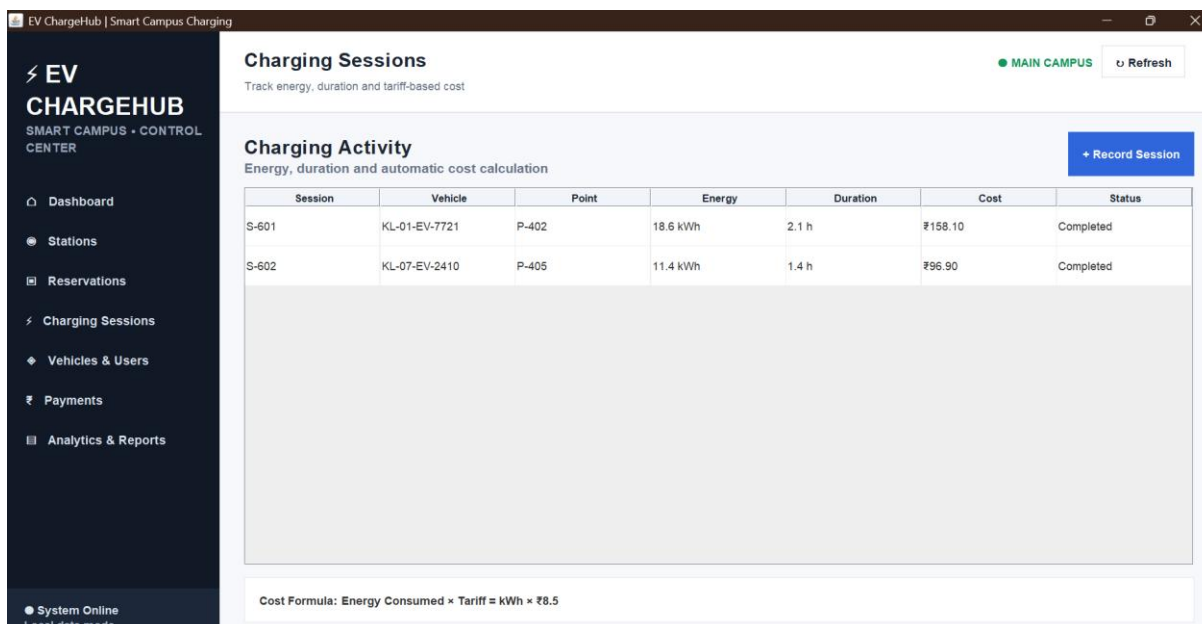


Figure 3. Charging Sessions – charging activity with energy, duration and calculated cost.

EV CHARGEHUB
SMART CAMPUS • CONTROL CENTER

Dashboard

Stations

Reservations

Charging Sessions

Vehicles & Users

Payments

Analytics & Reports

System Online

Local data mode

Reservations

Schedule charging slots without conflicts

MAIN CAMPUS Refresh

Reservation Desk

Book an available point and create a traceable slot record

New Reservation

ID	Vehicle	Charging Point	Slot	Status
R-501	KL-07-EV-2410	P-403	Today • 02:30 PM	Confirmed
R-502	KL-08-EV-5016	P-408	Today • 04:00 PM	Confirmed

Figure 4. Reservations – reservation desk with vehicle, point, slot and status.

EV CHARGEHUB

SMART CAMPUS • CONTROL CENTER

Dashboard

Stations

Reservations

Charging Sessions

Vehicles & Users

Payments

Analytics & Reports

System Online

Local data mode

Vehicles & Users

Campus account and EV registration

MAIN CAMPUS

Refresh

Campus Accounts & EVs

Identity and vehicle information used by reservations

Register User

Register Vehicle

User ID	Name	Email	Vehicle	Model	Registration
U-101	Ananya Menon	ananya@smartcampus.edu	EV-201	Tata Nexon EV	KL-07-EV-2410
U-102	Rahul Nair	rahul@smartcampus.edu	EV-202	MG ZS EV	KL-01-EV-7721
U-103	Meera Krishnan	meera@smartcampus.edu	EV-203	Hyundai Ioniq 5	KL-08-EV-5016

Figure 5. Vehicles & Users – user and registered EV information.

EV ChargeHub | Smart Campus Charging

EV CHARGEHUB

SMART CAMPUS • CONTROL CENTER

Dashboard

Stations

Reservations

Charging Sessions

Vehicles & Users

Payments

Analytics & Reports

System Online

Local data mode

Payments

Charging transactions and payment status

MAIN CAMPUS

Refresh

Payment Ledger

Transactions automatically created for completed sessions

Payment	Session	Vehicle	Amount	Status
PAY-701	S-601	KL-01-EV-7721	₹158.10	Paid
PAY-702	S-602	KL-07-EV-2410	₹96.90	Paid

Figure 6. Payments – payment ledger associated with completed charging sessions.

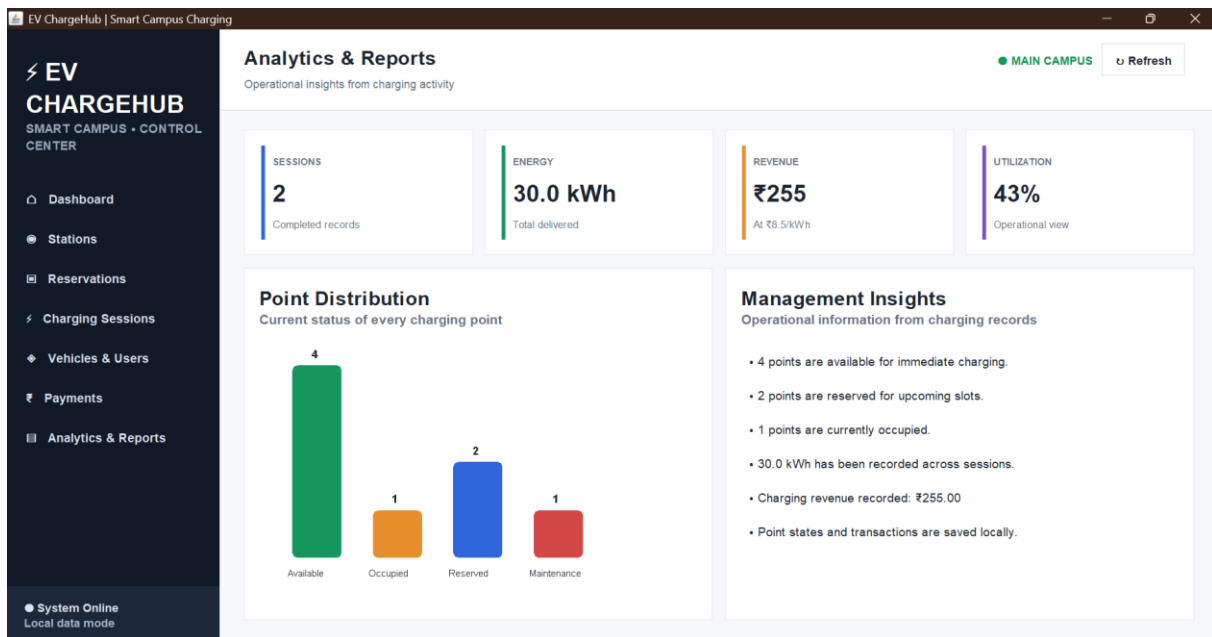


Figure 7. Analytics & Reports – KPI summary, point distribution and management insights.

TESTING AND VALIDATION

Testing was performed at the GUI and workflow level using the supplied application. The checks focus on navigation, data-entry validation, state changes, charging-cost calculation and visual output.

Test ID	Test Case	Expected Result	Observed Result
T01	Launch application	Dashboard opens successfully	Dashboard displayed with KPI cards and live point status
T02	Open Stations	Station directory is displayed	Station records and Change actions displayed
T03	Change point status	Selected point status is updated	Status can be changed and page refreshed
T04	Create reservation	Reservation is added for selected vehicle/point/slot	Reservation workflow is available
T05	Record session	Session and payment are created	Session and payment entries appear
T06	Invalid session values	Validation warning is shown	Numeric validation warning is implemented
T07	Register user/vehicle	New record is added	Registration dialogs are available
T08	Open Payments	Payment ledger is displayed	Payment records and amounts displayed
T09	Open Analytics	KPIs and chart reflect current records	Point distribution and management insights displayed
T10	Refresh/navigation	Current page is rebuilt	Navigation and refresh workflow operate through the GUI

COST CALCULATION

The Session constructor calculates cost as $\text{kWh} \times \text{tariff}$. With the configured tariff of ₹8.50 per kWh, the sample sessions of 18.6 kWh and 11.4 kWh correspond to ₹158.10 and ₹96.90 respectively. The total sample energy is 30.0 kWh and the corresponding total charging revenue is ₹255.00.

INDIVIDUAL CONTRIBUTION

My main contribution to the project was focused on the backend development, database management, JDBC connectivity, and reservation functionality of the EV Charging Management System. I worked on designing the MySQL database and creating the required tables for users, vehicles, charging stations, charging points, reservations, charging sessions, and payments, along with suitable primary and foreign key relationships. I also worked on connecting the Java application with MySQL using JDBC and implemented SQL operations such as inserting, retrieving, updating, and deleting records using Statement and PreparedStatement. I was involved in the User and Vehicle Management module, where user and vehicle details could be registered, searched, and updated. I also worked on the reservation module, including selecting charging stations and points, checking their availability, and preventing conflicting reservations. The charging-point status was maintained based on activities such as reservation and charging, with statuses including Available, Occupied, Reserved, and Maintenance. I also handled input validation and exception handling for situations such as duplicate vehicle numbers, missing details, unavailable charging points, incorrect reservation information, and database connection errors. During development, I tested and debugged the database operations to make sure that information was correctly stored, retrieved, and updated and that the backend worked properly with the GUI. Through this project, I gained practical experience in MySQL, JDBC, SQL queries, database design, Java programming, validation, exception handling, and backend development, and my work helped build the database and backend functionality of the overall EV Charging Management System.

CONCLUSION

EV ChargeHub is a practical desktop application designed to simplify EV charging management on a university campus. It brings charging-point monitoring, reservations, charging sessions, user and vehicle management, payments, and analytics together in one place. The dashboard provides a quick view of charging-point statuses, while the reports and analytics help understand energy consumption, charging activity, revenue, and station utilization. The implementation demonstrates important workflows such as reservation handling, charging-session management, status updates, cost calculation, and data visualization. Overall, the project shows how Java GUI, database concepts, event handling, and visualization can be combined to create a user-friendly smart-campus EV charging management solution.

FUTURE ENHANCEMENTS

- Integrate a persistent database for all users, vehicles, stations, reservations, sessions and payments.
- Introduce a dedicated backend/service layer for database operations.
- Add role-based authentication for charging-desk operators and administrators.
- Implement stronger date/time-based reservation conflict validation.
- Add richer daily and monthly energy and utilization analytics.
- Provide exportable reports and improved administrative monitoring.

SOURCE AND SUBMISSION NOTES

The report has been reformatted to follow the supplied reference document's presentation style: centered project title, simple academic headings, Times New Roman body text, page-separated pseudocode, diagram pages, implementation-code pages, output screenshots, testing, and individual contribution. The project content and screenshots were retained from the student's supplied Word report rather than replacing the project with unrelated material.